Отчёт по лабораторной работе

Потапов С. А. НКАбд-05-25

Потапов Савелий Александрович

Содержание

1 Цель работы 5

2 Задание 6

3 Выполнение лабораторной работы 7

4 Контрольные вопросы 14

5 Выводы 16

Список иллюстраций

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Используя команды getopts grep, написать командный файл, который анализирует командную строку с ключами: – -iinputfile — прочитать данные из указанного файла; – -ooutputfile — вывести данные в указанный файл; – -ршаблон — указать шаблон для поиска; – -C — различать большие и малые буквы; – -n — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом -p.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции exit(n), передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды $?, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до (например 1.tmp, 2.tmp, 3.tmp,4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find).

3 Выполнение лабораторной работы

search.sh - GNU Emacs at sapotapov

File Edit Options Buffers Tools Sh-Script Help

Save Undo

```
input_file=""
output_file=""
pattern=""
case_sensitive=false
show_line_numbers=false

while getopts "i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive=true ;;
        n) show_line_numbers=true ;;
    esac
done

grep_opts=""
if [ "$case_sensitive" = false ]; then
    grep_opts="$grep_opts -i"
fi
if [ "$show_line_numbers" = true ]; then
    grep_opts="$grep_opts -n"
fi
if [ -z "$output_file" ]; then
    grep $grep_opts -- "$pattern" "$input_file"
else
    grep $grep_opts -- "$pattern" "$input_file" > "$output_file"
fi
```

!

search.sh log.txt

```
this
is
a
test
error
test
end
```

!

```
(base) sapotapov@sapotapov:~/labworks_wip/lab13/report/codes$ bash search.sh -i log.txt -p error -n
6:error
```
!

Я написал программу, ищущую в файле искомую комбинацию символов. Сверху прикреплено содержание текстового файла, по которому осуществляется тестовый поиск. Как видно, программа наодит искомую комбинацию «error» в 6 строке.

Открыть

sign_check.sh
~/labworks_wip/lab13/report/codes

```
./sign_check

code=$?

case $code in
    0) echo "Введённое число равно нулю." ;;
    1) echo "Введённое число больше нуля." ;;
    2) echo "Введённое число меньше нуля." ;;
    *) echo "Неизвестный код завершения: $code" ;;
esac
```

!

Открыть

sign_check.c
~/labworks_wip/lab13/report/codes

sign_check.sh | sign_check.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;

    printf("Введите целое число: ");
    scanf("%d", &num);
    if (num == 0) {
        exit(0);
    } else if (num > 0) {
        exit(1);
    } else {
        exit(2);
    }
}
```

!

```
(base) sapotapov@sapotapov:~/labworks_wip/lab13/report/codes$ ./sign_check.sh
Введите целое число: 5
Введённое число больше нуля.
```

!

Я написал программу, определяющую, больше или меньше число относительно нуля.

Открыть

manage_files.sh
~/labworks_wip/lab13/report/codes

```
create_files() {
    local n=$1
    echo "Создаю $n файлов (1.tmp ... $n.tmp)..."
    for i in $(seq 1 $n); do
        touch "${i}.tmp"
        echo "  создан ${i}.tmp"
    done
    echo "Готово."
}

delete_files() {
    local n=$1
    echo "Удаляю файлы (1.tmp ... $n.tmp)..."
    for i in $(seq 1 $n); do
        if [ -f "${i}.tmp" ]; then
            rm "${i}.tmp"
            echo "  удалён ${i}.tmp"
        fi
    done
    echo "Готово."
}

N=$1
if [ $# -eq 2 ]; then
    case "$2" in
        -d|--delete)
            delete_files $N
            exit 1
            ;;
    esac
else
    # По умолчанию - создание файлов
    create_files $N
fi
```

!

```
(base) sapotapov@sapotapov:~/labworks_wip/lab13/report/codes$ bash manage_files.sh 5
Создаю 5 файлов (1.tmp ... 5.tmp)...
  создан 1.tmp
  создан 2.tmp
  создан 3.tmp
  создан 4.tmp
  создан 5.tmp
Готово.
```

!

```
(base) sapotapov@sapotapov:~/labworks_wip/lab13/report/codes$ bash manage_files.sh 5 -d
Удаляю файлы (1.tmp ... 5.tmp)...
  удалён 1.tmp
  удалён 2.tmp
  удалён 3.tmp
  удалён 4.tmp
  удалён 5.tmp
Готово.
```

!

Я написал программу, способную создавать и удалять определенное число временных файлов.

Открыть archive.sh ~/labworks_wip/lab13/report/codes

```
DIR="$1"
ARCHIVE="$2"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория '$DIR' не существует"
    exit 1
fi

cd "$DIR" || exit 1
find . -maxdepth 1 -type f -mtime -7 -print0 | tar -cvf "$ARCHIVE" --null -T
-

if [ $? -eq 0 ]; then
    echo "Архив '$ARCHIVE' создан (только файлы, изменённые за последнюю
неделю)"
else
    echo "Ошибка: либо нет подходящих файлов, либо проблемы с созданием
архива"
    exit 1
fi
```

!

```
(base) sapotapov@sapotapov:~/labworks_wip/lab13/report/codes$ bash archive.sh ~/labworks_wip/lab2 backup.tar
Архив 'backup.tar' создан (только файлы, изменённые за последнюю неделю)
```

!

Домашняя папка / labworks_wip / lab2 / backup

LICENSE

!

Я создал программу, архивирующую файлы, измененные за последнюю неделю. Я зашёл в папку, в которой выполнял вторую работу, ввёл случайную

комбинацию символов в LICENSE, сохранил файл и запустил программу. Как видно, в архив попал только упомянутый файл.

4 Контрольные вопросы

1) getopts — встроенная команда оболочки (bash), предназначенная для разбора опций (ключей) командной строки. Она позволяет обрабатывать короткие опции с аргументами (например, -f файл), автоматически перебирая их и устанавливая переменные.

2) Метасимволы (например, *, ?, [ ]) *используются в командной оболочке для генерации имён файлов. Оболочка заменяет шаблон, содержащий метасимволы, на список реальных имён файлов в файловой системе, которые соответствуют этому шаблону. Например,* .txt превращается во все файлы с расширением .txt в текущем каталоге.

3) Операторы управления действиями (управляющие операторы) в командной оболочке: && — логическое «И»: команда справа выполняется, только если команда слева завершилась успешно (код 0). || — логическое «ИЛИ»: команда справа выполняется, только если команда слева завершилась ошибкой (код ≠0). ; — последовательное выполнение команд (без проверки кода возврата). & — запуск команды в фоновом режиме. | — конвейер (передача вывода одной команды на ввод другой). () — выполнение команд в подоболочке. {} — группировка команд в текущей оболочке.

4) Операторы (команды) для прерывания цикла: break — немедленно завершает выполнение ближайшего вложенного цикла (for, while, until). continue — прерывает текущую итерацию цикла и переходит к следую-

щей (проверке условия или следующему элементу).

5) true и false — это стандартные команды Unix, которые ничего не делают, но возвращают определённый код завершения: true всегда возвращает 0 (успех), а false возвращает ненулевое значение (обычно 1, ошибка). Они часто используются в скриптах для создания бесконечных циклов (while true), условных проверок или как заглушки.

6) Строка if test -f man$s$/i.$s : test - f$—, $().man$s/i.s — путь, где переменные $s и $i подставляются. Например, если s=1, i=file, то путь man1/file.1. Вся конструкция проверяет, существует ли файл с таким именем; если да, то выполняется ветка then. Таким образом, скрипт проверяет наличие указанного файла перед дальнейшими действиями.

7) Различия между while и until: while повторяет тело цикла пока условие истинно (код возврата условия равен 0). Цикл выполняется до тех пор, пока условие даёт успех (true). until повторяет тело цикла пока условие ложно (код возврата условия не равен 0, т.е. ошибка).

5 Выводы

В процессе данной лабораторной работы я научился писать более сложные командные файлы с использованием логических управляющих конструкций и циклов. Эти навыки помогут мне в дальнейшем.